

实验一：手写数字识别

尧祥临 202518023406038 前沿交叉科学学院

1 任务介绍

本次实验的任务是在 MNIST 手写数字数据集上构造并训练一个卷积神经网络模型，实现对 0 到 9 共 10 类手写数字图象的分类。实验指导书要求完成 PyTorch 环境搭建、MNIST 数据加载、卷积神经网络组织结构构建、训练与评估，并使测试集准确率达到 98% 及以上。

MNIST 数据集由灰度手写数字图象组成，每张图象大小为 28×28 。本实验中使用 torchvision 提供的接口加载数据，训练集原始规模为 60000 张，测试集规模为 10000 张。为了观察训练过程中的泛化表现，我将原始训练集划分为 55000 张训练样本和 5000 张验证样本，测试集保持为 10000 张样本。所有输入图象只进行 ToTensor 转换，没有额外使用数据增强操作。

2 模型架构

为了匹配 MNIST 图象分辨率较小、类别数较少的特点，本实验没有采用过深的网络，而是实现了一个由两组卷积、激活和池化层组成的 CNN 模型。整体流程为

Input $\rightarrow$ Conv2d $\rightarrow$ ReLU $\rightarrow$ MaxPool
 $\rightarrow$ Conv2d $\rightarrow$ ReLU $\rightarrow$ MaxPool
 $\rightarrow$ Flatten $\rightarrow$ Linear $\rightarrow$ ReLU $\rightarrow$ Linear.

卷积层的输出尺寸可由下式计算：

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2P - K}{S} \right\rfloor + 1,$$

其中 H_{in} 表示输入尺寸， K 表示卷积核大小， P 表示 padding 大小， S 表示步长。本模型两个卷积层均采用 5×5 卷积核、步长为 1、padding 为 2，因此卷积操作本身保持特征图空间尺寸不变；随后通过 2×2 最大池化将空间尺寸减半。

表 1: CNN 模型结构

层	参数设置	输出尺寸	说明
Input	–	$1 \times 28 \times 28$	MNIST 灰度图象
Conv1	$1 \rightarrow 32, 5 \times 5$	$32 \times 28 \times 28$	padding 为 2, 保持分辨率
ReLU1	–	$32 \times 28 \times 28$	引入非线性表达能力
MaxPool1	2×2 , stride 2	$32 \times 14 \times 14$	下采样并增强局部平移鲁棒性
Conv2	$32 \rightarrow 64, 5 \times 5$	$64 \times 14 \times 14$	提取更高层次特征
ReLU2	–	$64 \times 14 \times 14$	非线性激活
MaxPool2	2×2 , stride 2	$64 \times 7 \times 7$	第二次下采样
Flatten	–	3136	将特征图展平成向量
Linear1	$3136 \rightarrow 128$	128	全连接隐藏层
Linear2	$128 \rightarrow 10$	10	输出 10 个类别的 logits

按照上述结构计算, 模型总参数数量为 454922。其中第一层卷积参数数量为 $1 \times 32 \times 5 \times 5 + 32 = 832$, 第二层卷积参数数量为 $32 \times 64 \times 5 \times 5 + 64 = 51264$, 两个全连接层参数数量分别为 401536 和 1290。可以看出, 主要参数集中在从 $64 \times 7 \times 7$ 特征图到 128 维隐藏表示的第一个全连接层中。

模型在代码中的核心定义如下:

```

1 class CNN(nn.Module):
2     def __init__(self, hidden_dim=128):
3         super(CNN, self).__init__()
4         self.conv1 = nn.Conv2d(1, 32, kernel_size=5, stride=1, padding=2)
5         self.relu1 = nn.ReLU()
6         self.pool1 = nn.MaxPool2d(kernel_size=2, stride=2)
7
8         self.conv2 = nn.Conv2d(32, 64, kernel_size=5, stride=1, padding=2)
9         self.relu2 = nn.ReLU()
10        self.pool2 = nn.MaxPool2d(kernel_size=2, stride=2)
11
12        self.fc = nn.Sequential(
13            nn.Flatten(),
14            nn.Linear(64 * 7 * 7, hidden_dim),
15            nn.ReLU(),
16            nn.Linear(hidden_dim, 10)
17        )
18
19    def forward(self, x):
20        x = self.pool1(self.relu1(self.conv1(x)))
21        x = self.pool2(self.relu2(self.conv2(x)))
22        return self.fc(x)

```

3 参数设置

本实验使用交叉熵损失函数训练多分类模型。设单个 batch 大小为 B ，第 i 个样本对应类别为 y_i ，模型输出 logits 为 z_i ，则损失函数为

$$\mathcal{L} = -\frac{1}{B} \sum_{i=1}^B \log \frac{\exp(z_{i,y_i})}{\sum_{c=0}^9 \exp(z_{i,c})}.$$

优化器采用 AdamW。相比普通随机梯度下降，AdamW 能够根据梯度一阶矩和二阶矩的估计自适应调整参数更新幅度，并将权重衰减与梯度更新解耦，对这种中小规模 CNN 训练较为稳定。具体超参数如表 2 所示。

表 2: 训练超参数设置

项目	设置
框架	PyTorch, torchvision
数据集	MNIST, 训练/验证/测试划分为 55000/5000/10000
输入预处理	<code>transforms.ToTensor()</code>
Batch size	64
Epochs	20
Learning rate	5×10^{-4}
隐藏层维度	128
损失函数	<code>nn.CrossEntropyLoss()</code>
优化器	<code>torch.optim.AdamW</code>
训练设备	CUDA GPU
模型保存路径	Lab1/model/cnn_mnist.pth
日志保存路径	Lab1/logs/train_log.csv

4 训练过程

训练阶段，模型首先进入 `train` 模式，对每个 batch 执行前向传播、损失计算、反向传播和参数更新。验证与测试阶段，模型进入 `eval` 模式，并通过 `torch.argmax` 在 10 个输出 logits 中选取预测类别。训练过程中每个 epoch 结束后都会分别计算验证集和测试集上的 loss 与 accuracy，并将结果写入 CSV 日志文件。

```

1 def train(model, train_loader, optimizer, criterion, device):
2     model.train()
3     total_loss = 0.0
4     total_samples = 0
5     for images, labels in train_loader:
6         images, labels = images.to(device), labels.to(device)

```

```

7     outputs = model(images)
8     loss = criterion(outputs, labels)
9     optimizer.zero_grad()
10    loss.backward()
11    optimizer.step()
12    total_loss += loss.item() * images.size(0)
13    total_samples += images.size(0)
14    return total_loss / total_samples
15
16 def evaluate(model, data_loader, criterion, device):
17     model.eval()
18     correct = 0
19     total = 0
20     total_loss = 0.0
21     for images, labels in data_loader:
22         images, labels = images.to(device), labels.to(device)
23         outputs = model(images)
24         preds = torch.argmax(outputs, dim=1)
25         loss = criterion(outputs, labels)
26         total_loss += loss.item() * images.size(0)
27         correct += (preds == labels).sum().item()
28         total += labels.size(0)
29     return total_loss / total, correct / total

```

表 3 给出了若干关键 epoch 的训练记录。可以看到，模型在第 1 个 epoch 结束时测试准确率已经达到 98.32%，超过实验要求的 98%；随着训练继续进行，训练损失持续下降，验证集和测试集准确率整体保持上升趋势，最终第 20 个 epoch 测试准确率达到 99.33%。

表 3: 训练过程中的 loss 与 accuracy

Epoch	Train Loss	Val Loss	Val Acc	Test Loss	Test Acc
1	0.2084	0.0680	97.78%	0.0518	98.32%
2	0.0580	0.0445	98.48%	0.0362	98.78%
5	0.0235	0.0384	98.80%	0.0321	98.86%
10	0.0087	0.0326	99.18%	0.0308	99.19%
15	0.0031	0.0319	99.14%	0.0297	99.23%
19	0.0026	0.0267	99.32%	0.0310	99.31%
20	0.0022	0.0289	99.20%	0.0312	99.33%

图 1 展示了训练损失和测试准确率随 epoch 变化的曲线。左图中训练损失在前几个 epoch 快速下降，随后逐渐收敛到接近 0 的水平；右图中测试准确率在第 1 个 epoch 后已经较高，之后在 99% 附近小幅波动并继续提高。第 16 个 epoch 附近验证损失和测试损失有一次上升，但后续又恢复到较低水平，说明模型整体没有出现持续恶化的过拟合趋势。

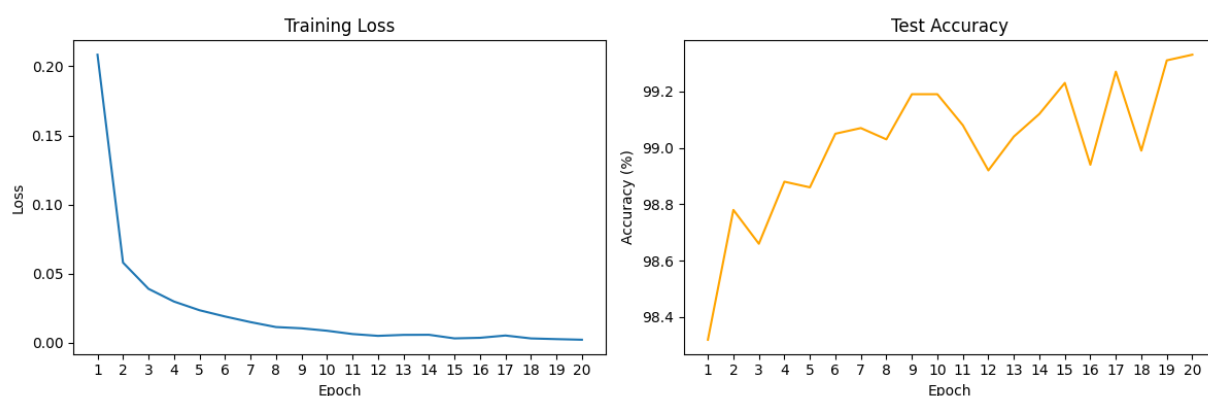


图 1: 训练损失与测试准确率变化曲线

从实验结果来看, 该模型能够较好地学习 MNIST 手写数字中的局部笔画特征。第一层卷积主要捕捉边缘和局部纹理, 第二层卷积在下采样后的特征图上组合更抽象的笔画模式, 最后通过全连接层完成类别判别。由于 MNIST 任务相对规整, 即使没有额外的数据增强和复杂网络结构, 两层卷积网络也能够达到较高准确率。

5 总结

本次实验基于 PyTorch 完成了 MNIST 手写数字识别任务, 主要工作包括数据集加载与划分、CNN 模型搭建、损失函数和优化器设置、训练日志记录、模型保存以及结果可视化。最终模型在测试集上取得 99.33% 的准确率, 满足实验指导书中测试集准确率达到 98% 及以上的要求。后续如果继续改进, 可以考虑固定随机种子以增强结果可复现性, 在输入端加入标准化处理, 或进一步补充混淆矩阵来分析各个数字类别之间的错误模式。